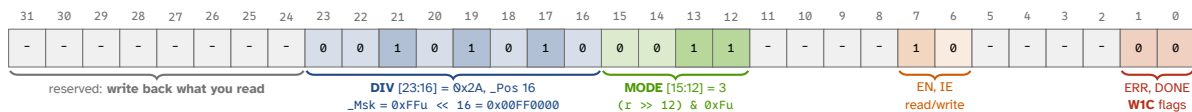


In one line: A register is a fixed address reached through a volatile `uint32_t *`; a field is a **position + width** — read it *shift*, then *mask*, write it *clear*, then *OR*, in **unsigned** arithmetic with shift counts below the width. `r |= m` is a load, an OR and a store: SET/CLR registers, bit-band aliases and write-1-to-clear flags exist so that one bit changes in **one store**.

CTRL — an illustrative 32-bit register holding `0x002A3080`; bit 31 left, bit 0 right. Field = *position + width*; a 1-bit flag is a field of width 1.



Six idioms

set <i>n</i>	<code>r = 1u << n</code>	OR forces a 1
clear <i>n</i>	<code>r &= ~(1u << n)</code>	AND with a hole
toggle <i>n</i>	<code>r ^= 1u << n</code>	XOR flips
test <i>n</i>	<code>(r >> n) & 1u</code>	0/1: !(r & m) too
get field	<code>(r & Msk) >> Pos</code>	Msk = shifted mask
put field	<code>r = (r & ~Msk) ((v << Pos) & Msk)</code>	clear, OR; the last & Msk stops a too-big v

Put **DIV** = **0x15** into `0x002A3080`: `r & ~0x00FF0000 = 0x00003080`; `(0x15 << 16) & Msk = 0x00150000`; OR $\rightarrow$ `0x00153080`.

	CMSIS (Arm)	ESP-IDF soc/	Linux
where	X_Pos	X_S	inside the mask
mask	X_Msk (shifted)	X_V plain, X_M shifted	GENMASK(h, l), BIT(n)
put	_VAL2FLD(X, v)	REG_SET_FIELD(r, X, v)	FIELD_PREP(m, v)
get	_FLD2VAL(X, r)	REG_GET_FIELD(r, X)	FIELD_GET(m, r)

FIELD_PREP rejects a non-constant mask, or a constant value too wide for it, at compile time.

Precedence — where bit tests break

`* / % + - << >> < <= >= == != & ^ | && || ?: = |=`

$\leftarrow$ binds tighter. `r & 1u << n == 0` parses as `r & ((1u << n) == 0)`:
parenthesise every mask test.

Shifts — C23 6.5.8

Operands promoted; result type = the promoted left one; a count < 0 or $\geq$ its width is **UB for every type**. C23's mandatory two's complement (6.2.6.2) changed none of it.

32-bit int	result
<code>1u << 31</code>	<code>0x80000000</code> ; unsigned $\ll$ wraps mod 2^N
<code>1 << 31</code>	UB : 2^{31} is not an int value (GCC defines it as an extension; <code>-fsanitize=shift</code> still reports it)
<code>-1 << 1</code>	UB : negative left operand
<code>-8 >> 1</code>	implementation-defined: GCC sign-extends $\rightarrow -4$
<code>x << 32, x >> 32</code>	UB for <code>uint32_t x</code> : <code>(1u << w) - 1</code> fails at <code>w = 32</code> ; <code>~0u >> (32 - w)</code> works for <code>w 1..32</code>
<code>uint64_t m = 1 << 40;</code> <code>(uint8_t)0x80 << 24</code>	UB : still an int shift — <code>1ULL << 40</code> promoted to int, $2^{31} \rightarrow$ UB : cast to <code>uint32_t</code> first

Constants, widths, printing

- Hex, octal, binary constants try int, unsigned, long, ...: `0x80000000` is unsigned int; decimal `2147483648` is a (long) long.
- C23: `0b1011'0100` (binary + ' separator); `printf("%b")` $\rightarrow$ `0b10110100` (glibc ≥ 2.35); `%w32x` prints a `uint32_t`; `UINT32_WIDTH = 32`.
- `uintN_t` is **optional** (defined when such a type exists); `uint_leastN_t` is required. `UINT8_C(1)` has the promoted type, int.
- `uint32_t` may be unsigned long: `ESP_IDF  $\geq$  5.0` (GCC 11) on Xtensa and RISC-V — "%u" warns, print with `PRId32`.
- Promotion**: `uint8_t x = 0xFF`; `~x` is the int $\rightarrow 256$, never 0 — write `(uint8_t)~x`. C23 unsigned `_BitInt(N)` is *not* promoted: `~` stays *N* bits (GCC 14+: `x86`, `AArch64`).

The lowest set bit

<code>x</code>	<code>0 1 0 1 0 1 0 0 0</code>	<code>0x58</code>
<code>x - 1</code>	<code>0 1 0 1 0 1 1 1 1</code>	<code>0x57</code> lowest 1 $\rightarrow$ 0, 0s below $\rightarrow$ 1
<code>x & (x - 1)</code>	<code>0 1 0 1 0 0 0 0 0</code>	<code>0x50</code> clears the lowest 1
<code>~x</code>	<code>1 0 1 0 1 0 1 0 0 0</code>	<code>0xA8</code> 2^8 — <i>x</i> (unsigned)
<code>x & ~x</code>	<code>0 0 0 0 0 1 0 0 0</code>	<code>0x08</code> isolates it
<code>x (x - 1)</code>	<code>0 1 0 1 1 1 1 1 1</code>	<code>0x5F</code> fills the trailing 0s
<code>x ^ (x - 1)</code>	<code>0 0 0 0 1 1 1 1 1</code>	<code>0x0F</code> mask up to it

- Loop `x &= x - 1` to 0 = popcount in $O(\text{set bits})$ (Wegner 1960; K&R ex. 2-9). One bit set: `x && !(x & (x - 1))` — forget `x &&` and 0 passes.
- $\alpha = 2^k$, unsigned `h`: `h & (a - 1) = h % a`; align up: `(x + a - 1) & ~(a - 1)`.
- Sign-extend a *w*-bit field ($w \leq 31$): `m = 1u << (w - 1), (int32_t)(x ^ m) - (int32_t)m` — no implementation-defined step.
- Rotate, defined for `n 0..31`: `(x << n) | (x >> (-n & 31)) (x >> (32 - n))`: UB at 0. GCC 15: `__builtin_stdcl_rotate_left`.

<stdbit.h> (C23 7.18) vs the builtins

need	C23	GCC / Clang	at 0
count 1s	<code>stdc_count_ones</code>	<code>__builtin_popcount</code>	0
trailing 0s	<code>stdc_trailing_zeros</code>	<code>__builtin_ctz</code>	32 · UB
leading 0s	<code>stdc_leading_zeros</code>	<code>__builtin_clz</code>	32 · UB
ffs: index+1	<code>stdc_first_trailing_one</code>	<code>__builtin_ffs</code>	0 · 0
one bit set?	<code>stdc_has_single_bit</code>	<code>x && !(x & (x - 1))</code>	false
bits needed	<code>stdc_bit_width</code>	<code>32 - clz</code>	0 · UB
power of 2	<code>stdc_bit_floor / _ceil</code>		0 / 1

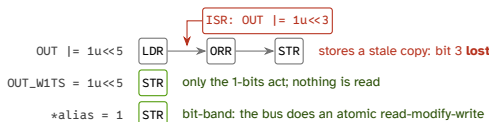
Suffixes `_uc...` `_ull` + type-generic (unsigned only); `bit_ceil` $\rightarrow$ 0 if 2^k does not fit. Shipped by the C library (glibc 2.39, Jan 2024); GCC 14: `__builtin_stdcl_*`, `__builtin_clzlg(x, 32)` (2nd arg = result for 0). Byte order: `__STDC_ENDIAN_NATIVE_...`.

Register access

```
#define REG32(a) (*(volatile uint32_t *) (a))
#define CTRL    REG32(PERIPH_BASE + 0x10u) // illustrative
#define DIV_Pos 16u
#define DIV_Msk (0xFFu << DIV_Pos)
uint32_t div = (CTRL & DIV_Msk) >> DIV_Pos; // 1 load
CTRL = (CTRL & ~DIV_Msk) | ((v << DIV_Pos) & DIV_Msk);
STATUS = DONE; // W1C: write only that 1
```

volatile: each access in the source happens; *what* counts as one is implementation-defined (6.7.4.1). Read-only register: `const volatile`. No atomicity.

One bit, one store



- ESP32** `GPIO_OUT_W1TS / _W1TC` (offset `0x08 / 0x0C`) set / clear the 1-bits. ESP-IDF's `REG_SET_BIT`, `REG_SET_FIELD` are plain C read-modify-writes.
- STM32** `GPIOx_BSRR`: bits 0–15 set, 16–31 reset ODR — “no risk of an IRQ occurring between the read and the modify access”; both: set wins.
- Bit-band** (Cortex-M3/M4 option; not M0/M0+, ARMv8-M): alias = base + `byte_offset` $\times 32$ + `bit` $\times 4$; 1MB of SRAM `0x2000_0000` $\rightarrow$ `0x2200_0000`, of peripherals `0x4000_0000` $\rightarrow$ `0x4200_0000`; `0x2200001C` = bit 7 of `0x20000000`. Bit 0 of the written word is the bit.

Status flags — how a bit is cleared

RAW	<code>0 0 1 0 0 1 0 1</code>	event latched, enabled or not
ENA	<code>0 0 0 0 0 1 1 1</code>	which events may interrupt
ST	<code>0 0 0 0 0 1 0 1</code>	= RAW & ENA, read-only: what the ISR reads
CLR	<code>0 0 0 0 0 1 0 0</code>	write 1 $\rightarrow$ that RAW bit clears (ESP32 + _INT_CLR)
kind	clears by	write
rc_w1	writing 1 (W1C): <code>EXTL_PR</code> , <code>ESP32 STATUS_W1TC</code>	<code>REG = F</code> ; <code>REG = F</code> writes back every pending 1 and clears flags never handled
rc_w0	writing 0: <code>STM32 TIMx_SR</code>	<code>SR = ~F</code> ; <code>SR &= ~F</code> zeroes flags raised between its load and store
rc_r	being read	any read is a side effect

C bit-fields vs hardware

Unit, order in it, straddling and a plain int field's sign are **implementation-defined** (GCC: the ABI decides; int fields signed); no & of a field; :0 closes the unit. GCC: volatile bit-fields “may be implicitly read when written to, or when adjacent bit-fields are accessed” — “unwise” for hardware; `-fstrict-volatile-bitfields` (default: the ABI's) forces one access of the declared width. ESP-IDF's `soc/*_struct.h` uses them anyway.

Remember

OR sets, AND-NOT clears, XOR flips, shift-then-mask reads; one bit, one store.